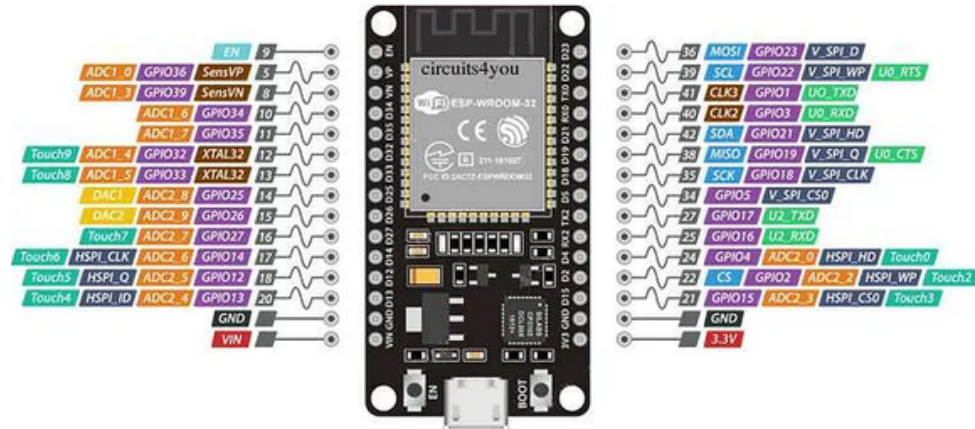




ESP32 Dev. Board Pinout

อธิบายส่วนต่าง ๆ ของ Pinout

- พินพาวเวอร์ (Power)
 - 3.3V - ให้แรงดันไฟฟ้า 3.3V สำหรับวงจรภายนอก
 - VIN - ใช้สำหรับจ่ายไฟให้บอร์ด (รองรับแรงดัน 5V)
 - GND - พินกราวด์ (ต่อลบ)
- พินอนาล็อก (ADC - Analog to Digital Converter) ใช้สำหรับอ่านค่าสัญญาณอะนาล็อก พินที่รองรับ ADC เช่น GPIO36, GPIO39, GPIO34, GPIO35 ฯลฯ
- พินดิจิทัล (GPIO - General Purpose Input Output) พินที่สามารถตั้งค่าเป็นอินพุตหรือเอาต์พุตได้ เช่น GPIO0, GPIO2, GPIO4 ฯลฯ
- พินสัมผัส (Touch Sensor) ใช้ในการตรวจจับสัมผัสโดยไม่ต้องใช้ปุ่มกด พินที่รองรับการสัมผัส เช่น Touch0 - Touch8 (GPIO4, GPIO0, GPIO2 ฯลฯ)
- พิน Digital to Analog Converter (DAC) ใช้สำหรับสร้างสัญญาณอะนาล็อกจากดิจิทัล GPIO25 (DAC1) และ GPIO26 (DAC2)
- พิน UART (Serial Communication) ใช้สำหรับสื่อสารแบบอนุกรม เช่น การเชื่อมต่อกับคอมพิวเตอร์หรืออุปกรณ์อื่น ๆ U0_TX (GPIO1) และ U0_RX (GPIO3) U1_TX (GPIO10) และ U1_RX (GPIO9)
- พิน SPI (Serial Peripheral Interface) ใช้สำหรับสื่อสารกับอุปกรณ์ที่รองรับ SPI เช่น เซ็นเซอร์, จอแสดงผล, SD Card เป็นต้น MOSI (GPIO23), MISO (GPIO19), SCK (GPIO18), CS (GPIO5)
- พิน I2C (Inter-Integrated Circuit) ใช้สำหรับเชื่อมต่ออุปกรณ์ I2C เช่น เซ็นเซอร์, จอ OLED, RTC เป็นต้น SDA (GPIO21) และ SCL (GPIO22)
- พินควบคุมพิเศษ
 - EN (Enable) GPIO36 - ใช้เปิด/ปิดการทำงานของ ESP32
 - BOOT (GPIO0) - ใช้บูตโหลดเฟิร์มแวร์เมื่อกดปุ่ม

Joystick

โพเทนชิโอเมเตอร์: หัวใจของจอยสติค

การทำงาน

โพเทนชิโอเมเตอร์ 2 ตัวทำงานแยกกัน ตัวหนึ่งจะรับการเคลื่อนที่ของจอยซ้าย-ขวา (แกน X) อีกตัวจะรับการเคลื่อนที่ของจอยขึ้น-ลง (แกน Y)

เมื่อเคลื่อนที่ของจอยสติคในแนวแกน X โพเทนชิโอเมเตอร์ทั้งสองจะทำงานพร้อมกัน โดยการเคลื่อนที่ของแกน X มีสัญญาณขาออกและขาอินพุตทั้งคู่

สวิตช์กดในลำ

จอยสติคมีสวิตช์กดที่ทำงานเมื่อกดลงบนปุ่มจอยสติค มันมีแค่ 1 ขั้วจะกดสวิตช์นี้ว่าจอยสติคกดอยู่ด้านล่างหรือไม่

สวิตช์นี้ต้องใช้ pull-up resistor เพื่อให้ทำงานได้อย่างถูกต้อง

Joystick Pin	ESP32 Pin	ประเภทสัญญาณ
VCC	3.3V	
GND	GND	
VRx	GPIO 34	Analog (ADC)
VRy	GPIO 35	Analog (ADC)
SW	GPIO 32	Digital (Input Pull-up)

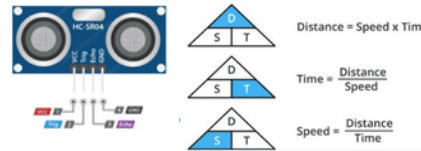
โมดูลจอยสติค	Arduino
GND	GND
VCC	5V
VRx	A0
VRy	A1
SW	8

การเคลื่อนที่แนวแกน X
เมื่อเอียงซ้าย-ขวา
เพลาแกน X จะหมุน

การเคลื่อนที่แนวแกน Y
เมื่อเอียงขึ้น-ลง เพลาแกน Y จะหมุน

การเคลื่อนที่แนวแกน Z
เพลาทั้งสองหมุนพร้อมกัน

Ultra sensor : HC-SR04 การวัดระยะทาง



HC-SR04 Pin	ESP32 Pin	คำแนะนำ
VCC	5V (หรือ VIN)	ใช้ไฟ 5V เพื่อความแม่นยำของ Sensor
GND	GND	ใช้ GND ร่วม
Trig	GPIO 5	ขาส่งสัญญาณ
Echo	GPIO 18	ขารับสัญญาณ (ใช้ Logic 3.3V ได้)

$$Distance = \frac{Duration \times 0.0343}{2}$$

ตัวอย่าง: สมมติว่าเราได้รับพัลส์ที่ใช้เวลา 500 ไมโครวินาทีบนขา Echo

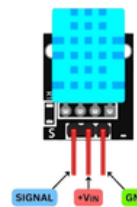
- ระยะทาง = ความเร็ว × เวลา
- ความเร็วของเสียง = 0.034 ซม./ไมโครวินาที
- ระยะทาง = 0.034 × 500 = 17 ซม.
- แต่ต้องหารด้วย 2 (เพราะเป็นระยะทางไป-กลับ)
- ระยะทางจริง = 17 ÷ 2 = 8.5 ซม.

ระยะทางที่วัดได้ 2-400 cm

ข้อมูลทางเทคนิคและคุณสมบัติ

0-50	±2.0	20-80	5%
ช่วงอุณหภูมิ ไม่ต่ำกว่า 0°C ถึง 50°C	ความแม่นยำอุณหภูมิ ความแม่นยำ ±2.0°C	ช่วงการขึ้น ไม่ต่ำกว่า 20% ถึง 80%	ความแม่นยำทางขึ้น ความแม่นยำ ±5%
	1Hz	3.3-5.5V	
อัตราการสุ่มตัวอย่าง ข้อมูลได้ทุก 1 วินาที		แรงดันไฟฟ้า ทำงานได้ 3.3V ถึง 5.5V	

DHT 11 วัดอุณหภูมิและความชื้น



- วัดอุณหภูมิ 0°C ถึง 50°C ความแม่นยำ ±2.0°C
- วัดความชื้น 20% ถึง 80% ความแม่นยำ 5%
- อัตราการสุ่มตัวอย่าง 1Hz (ข้อมูลใหม่ทุกวินาที)

แปลงหน่วย

แปลงอุณหภูมิจากเซลเซียสเป็นฟาเรนไฮต์โดยใช้สูตร: $T(^{\circ}F) = T(^{\circ}C) \times 9/5 + 32$

ส่วนประกอบวัดความชื้น

มีอิเล็กทรอนิกส์สองตัวพร้อมสารรองรับที่เก็บความชื้น (เกลือหรือพอลิเมอร์ พลาสติกน้ำไฟฟ้า) อยู่ตรงกลาง เมื่อความชื้นเพิ่มขึ้น สารรองรับจะดูดซับไอน้ำ ทำให้ความต้านทานระหว่างขั้วอิเล็กทรอนิกส์ลดลง

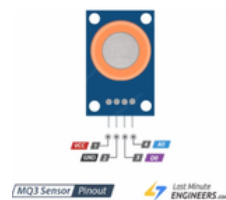
เทอร์มิสเตอร์ NTC

ใช้สำหรับวัดอุณหภูมิ เป็นตัวต้านทานที่ความต้านทานเปลี่ยนแปลงตามอุณหภูมิ "NTC" หมายถึง "Negative Temperature Coefficient" ซึ่งหมายความว่าความต้านทานลดลงเมื่ออุณหภูมิสูงขึ้น

ไอซี 8-bit SOIC-14

วัดและประมวลผลสัญญาณแอนะล็อกโดยใช้ค่าสัมประสิทธิ์การปรับเทียบที่เก็บไว้ แปลงสัญญาณแอนะล็อกเป็นดิจิทัล และส่งออกสัญญาณดิจิทัลที่มีข้อมูลอุณหภูมิและความชื้น

MQ-3 เซนเซอร์วัดแอลกอฮอล์



การเชื่อมต่อพิน

- VCC - จ่ายไฟให้โมดูล เชื่อมต่อกับ 5V ของ Arduino
- GND - พินกราวด์
- D0 - เอาต์พุตดิจิทัล (LOW เมื่อเกินค่าเกณฑ์)
- A0 - เอาต์พุตแอนะล็อกตามความเข้มข้น

ข้อมูลจำเพาะ

- แรงดันทำงาน: 5V
- ความต้านทานโหลด: 200 KΩ
- ช่วงความเข้มข้น: 25-500 ppm
- เวลาอุ่นเครื่อง: มากกว่า 24 ชั่วโมง

การปรับเทียบเซนเซอร์ MQ3



ข้อดีของ MQ3

- ใช้งานง่าย มีเอาต์พุตสองแบบ
- ความไวสูง ช่วง 25-500 ppm
- ปรับความไวได้ด้วยโพเทนชิโอเมเตอร์
- เหมาะสำหรับโปรเจกต์ DIY

ข้อควรระวัง

- ต้องอุ่นเครื่อง 24-48 ชั่วโมงครั้งแรก
- อุณหภูมิ 5-10 นาฬิกาใช้งานปกติ
- อย่าให้แอลกอฮอล์ของเหลวโดนเซนเซอร์
- ใช้เฉพาะแอลกอฮอล์เท่านั้น

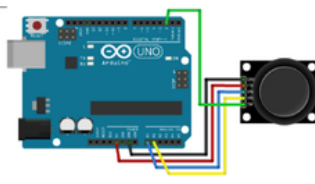
การเขียนโปรแกรม

- อ่านค่าแอนะล็อกด้วย analogRead()
- อ่านค่าดิจิทัลด้วย digitalRead()
- ตั้งค่าเกณฑ์ตามการทดสอบจริง
- ใช้ delay() เพื่อรอการอ่านค่าครั้งถัดไป

สูตร Potentiometer

$$ledVal = \frac{potVal \times 255}{1023}$$

ภาพที่ 3 ก็เหมือนไดอะแกรม Arduino UNO เชื่อมต่อกับ joystick ตามภาพ โดยมีวิธีในโปรแกรมที่อ่านค่า VRx, VRy และ SW ของ joystick ได้ด้วยคุณต๋อง



0 ถ้าเลือก joystick ไม่ผ่านค่าของบัสเซ็นเซอร์ค่า VRx และ VRy ได้คุณต๋อง
ดู เมื่อเลือก joystick ไม่ผ่านค่าของบัสเซ็นเซอร์ค่า VRx และ VRy ได้ 1023 ที่ข้อ หากปล่อย joystick ให้อยู่ตำแหน่งปกติตรงกลาง ค่า VRx และ VRy จะเป็นค่า 0

ii) หากมีอินเตอร์เฟซไฟฟ้าน +5V ของ joystick ไปกับ pin 3.3V ของ Arduino UNO ค่าที่อ่านได้จากเซ็นเซอร์เซ็นเซอร์ กลาง และ ด้านขวา จะเปลี่ยนแปลง หรือมี ค่าอื่น

3) ถ้ากด joystick ไม่อยู่ตำแหน่งกึ่งกลางตรงกลาง ค่า VRx และ VRy จะเป็นค่าอะไร

ที่ตำแหน่งกึ่งกลาง
potentiometer ที่สองจะอยู่ประมาณครึ่งหนึ่งของช่วงแรงดัน

ค่าของบัสเซ็นเซอร์ประมาณ

เมื่อแปลงเป็นค่า ADC 10 บิตของ Arduino:

$$\frac{2.5}{5} \times 1023 \approx 511.5$$

ค่าของบัสเซ็นเซอร์ประมาณ

$$VRx \approx 512, VRy \approx 512$$

คำตอบ

$$VRx \approx 512, VRy \approx 512$$

อาจคาดเคลื่อนเล็กน้อย เช่น 510, 515, 520 ได้ เพราะข้อจำกัดของไมโคร 100%

สรุปคำตอบแบบพร้อมเขียนส่ง

ข้อ ii) ถ้าเลือก joystick ไม่ผ่านบัสเซ็นเซอร์แล้วค่า VRx และ VRy เป็น 0 ทั้งคู่ ถือว่า ไม่ปกติและแบบเป็นไปไม่ได้ เพราะ joystick มี 2 แกน โดยเมื่อไม่ผ่านบัสเซ็นเซอร์ แกนหนึ่งควรมีค่าค่า แต่อีกแกนควรมีค่าสูง ไม่ควรเป็น 0 พร้อมกันทั้งสองแกน

ถ้าไม่ขยับ joystick แล้วอ่านได้ 1023 ทั้งสองแกน แสดงว่า ผิดปกติ เพราะปกติเมื่ออยู่กึ่งกลางควรอ่านได้ประมาณค่ากลาง ไม่ใช่ค่าสูงสุด เมื่อ joystick อยู่ตรงกลาง ค่าโดยประมาณคือ $VRx \approx 512, VRy \approx 512$

ข้อ ii)

ถ้านำสายไฟเลี้ยงของ joystick จาก 5V ไปต่อที่ 3.3V ค่า analog ที่อ่านได้จะ เปลี่ยน เพราะแรงดันสูงสุดที่ออกจาก VRx/VRy จะลดลงจาก 5V เหลือ 3.3V ขณะที่ Arduino UNO ยังอ้างอิง ADC ที่ 5V ทำให้ช่วงค่าที่อ่านได้ลดลงจาก 0-1023 เหลือประมาณ 0-675 และค่ากลางจะเหลือประมาณ 338 ดังนั้นค่าซ้าย กลาง ขวา และบน ล่าง จะลดลงทั้งหมดเมื่อเทียบกับตอนจ่ายไฟ 5V

Ultrasonic Sensor (เซ็นเซอร์อัลตราโซนิก เช่น HC-SR04)

- หลักการทำงานของ Ultrasonic Sensor
 - การส่งและรับคลื่นเสียง: Ultrasonic Sensor จะส่งคลื่นเสียงอัลตราโซนิกออกไป (โดยทั่วไปความถี่ประมาณ 40 kHz) ผ่านตัวส่ง(transmitter) เมื่อคลื่นเสียงชนวัตถุแล้วสะท้อนกลับมายังตัวรับ(receiver)
 - การคำนวณระยะทาง:
 - เซ็นเซอร์จะวัดระยะเวลาที่ใช้ในการส่งออกและรับคลื่นเสียง เมื่อได้เวลานี้ เราจะคำนวณระยะทางโดยใช้สูตร: $\text{Distance} = \text{Speed of Sound} \times \text{Time} / 2$ (โดยที่ $\text{Time} = \frac{\text{Distance}}{\text{Speed of Sound}}$)
 - Speed of Sound: ความเร็วของเสียงในอากาศประมาณ 340 m/s (หรือประมาณ 0.0343cm/ μ s)
 - Time: เวลาที่ใช้ในการเดินทางไปและกลับของคลื่นเสียง (ในหน่วยไมโครวินาที) การหารด้วย 2 เนื่องจากคลื่นเสียงต้องเดินทางไปยังวัตถุและกลับมามากอีกครั้ง
- การเชื่อมต่อ Ultrasonic Sensor กับ ESP32
 - สายเชื่อมต่อหลัก:
 - TRIG (Trigger): เชื่อมต่อกับพินดิจิทัลบน ESP32 (ตัวอย่างใช้พิน 5)
 - ECHO (Echo): เชื่อมต่อกับพินดิจิทัลของ ESP32 (ตัวอย่างใช้พิน 18)หมายเหตุ: เนื่องจาก HC-SR04 ใช้แรงดัน 5V และ ESP32 รองรับ 3.3V ในการอ่านสัญญาณ ให้ใช้ตัวแบ่งแรงดัน (voltage divider) กับสาย ECHO หากจำเป็น
 - VCC: จ่ายไฟ 5V (ตรวจสอบว่าแหล่งจ่ายไฟของคุณรองรับ)
 - GND: ต่อสายกราวด์
- ตัวอย่างโค้ดสำหรับวัดระยะทางด้วย ESP32

คำอธิบายโค้ด:

- โค้ดนี้เริ่มต้นด้วยการตั้งค่า TRIG เป็น OUTPUT และ ECHO เป็น INPUT
- ส่งสัญญาณ pulse ไปที่ TRIG เพื่อเริ่มต้นการส่งคลื่นเสียง
- ใช้ฟังก์ชัน pulseIn() วัดเวลาที่สัญญาณ echo อยู่ในสถานะ HIGH
- คำนวณระยะทางจากเวลาที่วัดได้และแสดงผลผ่าน Serial Monitor

4. ข้อจำกัดของ Ultrasonic Sensor

- ช่วงการวัดระยะ:
 - HC-SR04 สามารถวัดระยะทางได้ประมาณ 2 cm ถึง 400 cm
 - ข้อจำกัด: หากวัตถุอยู่ใกล้เกินไป (น้อยกว่า 2 cm) หรือห่างเกินไป (มากกว่า 400 cm) ผลลัพธ์อาจไม่แม่นยำ
- การสะท้อนของคลื่นเสียง:
 - วัตถุที่มีพื้นผิวเรียบหรือไม่เรียบอาจทำให้คลื่นเสียงกระจัดกระจาย ส่งผลให้เกิดสัญญาณสะท้อนที่อ่อนหรือไม่ชัดเจน
 - สิ่งแวดล้อม เช่น ผนังหรือวัตถุที่อยู่ใกล้เคียง อาจทำให้เกิดสัญญาณสะท้อนที่ทำให้การวัดระยะทางคลาดเคลื่อนได้
- ผลกระทบจากสภาพแวดล้อม:
 - อุณหภูมิและความชื้นในอากาศมีผลต่อความเร็วของเสียง ซึ่งอาจเปลี่ยนแปลงค่าที่คำนวณได้
 - การรบกวนจากเสียงรอบข้างหรือคลื่นสะท้อนจากพื้นผิวที่ไม่ต้องการก็อาจส่งผลต่อความแม่นยำ
- ความละเอียดและความถูกต้อง:
 - ความละเอียดในการวัดเวลาของฟังก์ชัน pulseIn() มีผลโดยตรงต่อความแม่นยำของการวัดระยะทาง
 - ในบางกรณี วัตถุที่มีหลายชิ้นหรือหลายพื้นผิวในแนวเดียวกันอาจทำให้เกิดเซ็นเซอร์รับสัญญาณ echo สัญญาณ ซึ่งอาจทำให้ผลลัพธ์ผิดเพี้ยน

สรุปUltrasonic Sensor เช่น HC-SR04 ใช้หลักการส่งและรับคลื่นเสียงในการวัดระยะทางจากเวลาที่คลื่นสะท้อนกลับมาโดยคำนวณระยะทางด้วยสูตรที่แบ่งด้วย 2 เนื่องจากคลื่นต้องเดินทางไปและกลับ ขณะเดียวกันการเชื่อมต่อกับ ESP32ต้องระวังเรื่องแรงดันที่ใช้กับพิน ECHO และควรวางในสภาพแวดล้อมที่เหมาะสมเพื่อให้ได้ผลการวัดที่แม่นยำที่สุดDHT (เช่น DHT11, DHT22 - เซ็นเซอร์วัดอุณหภูมิและความชื้น)

DHT (เช่น DHT11, DHT22 - เซ็นเซอร์วัดอุณหภูมิและความชื้น)

- หลักการทำงานของเซ็นเซอร์ DHT
 - การวัดอุณหภูมิและความชื้น:เซ็นเซอร์ DHT ประกอบด้วยส่วนวัดอุณหภูมิและส่วนวัดความชื้นในอากาศ โดยใช้ตัวแปลงสัญญาณภายในเพื่อส่งออกข้อมูลในรูปแบบดิจิทัลผ่านทางสัญญาณแบบ one-wire
 - การสื่อสารข้อมูล:หลังจากที่ไม่โครคอนโทรลเลอร์ส่งสัญญาณเริ่มต้น (start signal) ให้กับเซ็นเซอร์ DHT เซ็นเซอร์จะส่งข้อมูลที่ประกอบด้วยข้อมูลความชื้นและอุณหภูมิเป็นชุดของไบนารี โดยมี byte สำหรับ checksum เพื่อยืนยันความถูกต้องของข้อมูล
- การเชื่อมต่อ DHT กับ ESP32
 - สายที่ใช้เชื่อมต่อ:
 - VCC: จ่ายไฟ 3.3V หรือ 5V (ตรวจสอบรุ่นของเซ็นเซอร์ว่าเหมาะกับแรงดันไหน)
 - GND: ต่อกราวด์
 - DATA: สายข้อมูลที่เชื่อมต่อกับพินดิจิทัลของ ESP32หมายเหตุ: ควรต่อ resistor pull-up (มักใช้ 4.7k Ω ถึง 10k Ω) ระหว่างสาย DATA กับ VCC เพื่อให้สัญญาณมีควมฉิ่ง
- รูปแบบของข้อมูลที่ได้รับจาก DHT
 - ข้อมูลที่ส่งออก:เซ็นเซอร์ DHT จะส่งข้อมูลในรูปแบบ 5 ไบนารี ซึ่งประกอบไปด้วย:
 - 0 2 ไบนารีแรก: ค่าความชื้น (อาจเป็นแบบ integer หรือ float ขึ้นอยู่กับรุ่น)
 - 0 2 ไบนารีถัดมา: ค่าอุณหภูมิ
 - 0 1 ไบนารีสุดท้าย: Checksum สำหรับตรวจสอบความถูกต้องของข้อมูล

5. การใช้ไลบรารี DHT ใน Arduino IDE และ MicroPython

- Arduino IDE:
 - ติดตั้งไลบรารี DHT (เช่น "DHT sensor library" จาก Adafruit) ผ่าน Library Manager
 - ไลบรารีนี้จะช่วยจัดการการสื่อสารระหว่าง ESP32 กับเซ็นเซอร์ DHT โดยอัตโนมัติ
- MicroPython:
 - MicroPython บน ESP32 มักมีโมดูลสำหรับเซ็นเซอร์ DHT อยู่แล้ว หากไม่สามารถติดตั้งได้ผ่าน package manager ก็รองรับ
 - โมดูล dht ใน MicroPython ใช้งานง่ายและมีฟังก์ชัน measure(), temperature(), และhumidity()
- เปรียบเทียบ DHT11 vs DHT22 (ข้อดีข้อเสีย)DHT11
 - ข้อดี:
 - ราคาถูก เหมาะสำหรับโปรเจกต์ที่มีงบประมาณจำกัด
 - ใช้งานง่ายและเพียงพอต่อการใช้งานในงานที่ไม่ต้องการความแม่นยำสูง
 - ข้อเสีย:
 - ช่วงวัดความชื้นจำกัด (ประมาณ 20-90% RH)
 - ช่วงวัดอุณหภูมิถึงจำกัด (ประมาณ 0-50°C)
 - ความแม่นยำต่ำกว่าและอัปเดตข้อมูลช้ากว่า (อ่านได้ประมาณทุก 1-2 วินาที)DHT22
 - ข้อดี:
 - วัดได้ในช่วงที่กว้างกว่า (ความชื้น: 0-100% RH, อุณหภูมิ: -40 ถึง 80°C)
 - ความแม่นยำสูงกว่า เหมาะสำหรับงานที่ต้องการความละเอียดและการวัดที่แม่นยำ
 - ข้อเสีย:
 - ราคาสูงกว่า DHT11 เล็กน้อย
 - เวลาในการอ่านข้อมูลอาจช้ากว่าเล็กน้อยเมื่อเทียบกับ DHT11

ตัวอย่างเปรียบเทียบการเลือกใช้งาน:

- หากโปรเจกต์ของคุณเป็นงานพื้นฐานเช่นสมาร์โฮมหรือการแสดงผลข้อมูลในสภาพแวดล้อมที่ไม่ซับซ้อนDHT11 อาจเพียงพอ
- แต่ถ้าคุณต้องการความแม่นยำและการวัดที่ครอบคลุมช่วงอุณหภูมิและความชื้นที่กว้าง DHT22 จะเป็นตัวเลือกที่เหมาะสมกว่า

สรุปแล้ว เซ็นเซอร์ DHT (DHT11 และ DHT22) ใช้สำหรับวัดอุณหภูมิและความชื้นผ่านการสื่อสารด้วยสัญญาณดิจิทัล การเชื่อมต่อกับ ESP32 นั้นทำได้ง่ายผ่านสาย VCC, GND และ DATA โดยต้องคำนึงถึงการใส่ resistor pull-up ในสาย DATA ตัวอย่างโค้ดใน Arduino IDE และ MicroPython ที่แสดงข้างต้นจะช่วยให้คุณเริ่มต้นใช้งานได้ในทันที นอกจากนี้ การเปรียบเทียบระหว่าง DHT11 กับ DHT22 จะช่วยให้คุณเลือกเซ็นเซอร์ที่ตรงกับความต้องการของโปรเจกต์มากที่สุด

Motor Driver (เช่น L298N, L293D)1. Motor Driver (เช่น L298N, L293D) และหลักการทำงาน

- หน้าที่หลัก:Motor Driver ทำหน้าที่เป็นอินเวอร์เตอร์ระหว่างไมโครคอนโทรลเลอร์ (เช่น ESP32) กับมอเตอร์ DC ที่ต้องการแรงดันและกระแสสูงกว่าโดยตรงจากบอร์ดควบคุม
- การแปลงสัญญาณ: รับสัญญาณควบคุม (HIGH/LOW หรือ PWM) จาก ESP32 แล้วแปลงเป็นสัญญาณที่มอเตอร์ต้องการ
- ความสามารถ:ฟังก์ชันสำหรับสลับทิศทาง (Forward/Reverse) โดยใช้วงจร H-Bridge และสามารถควบคุมความเร็วได้ด้วยการใช้ PWM

2. H-Bridge คืออะไรและทำงานอย่างไร

- โครงสร้างของ H-Bridge:วงจร H-Bridge ประกอบด้วยสวิตช์ (หรือทรานซิสเตอร์) 4 ตัว จัดเรียงเป็นรูปตัว "H"
- การทำงาน:
 - เมื่อเปิดสวิตช์คู่แนวแยง (เช่นด้านบนซ้ายและด้านล่างขวา) กระแสจะไหลในทิศทางหนึ่ง ทาให้มอเตอร์หมุนไปด้านหน้า
 - เมื่อสลับการเปิดสวิตช์เป็นคู่อีกชุด (ด้านบนขวาและด้านล่างซ้าย) กระแสจะไหลกลับด้าน ทาให้มอเตอร์หมุนย้อนกลับ
 - ข้อดี:สามารถควบคุมทิศทางและความเร็วของมอเตอร์ได้อย่างมีประสิทธิภาพ

3. การควบคุมมอเตอร์ DC ด้วย Motor Driver

- การส่งสัญญาณควบคุม:ESP32 จะส่งสัญญาณควบคุม (เช่น digital output สำหรับเลือกทิศทาง และ PWM สำหรับควบคุมความเร็ว) ไปยัง Motor Driver
- การทำงานร่วมกับ H-Bridge:Motor Driver จะใช้วงจร H-Bridge เพื่อเปลี่ยนทิศทางกาไหลของกระแสในมอเตอร์ ทาให้สามารถหมุนไปในทิศทางที่ต้องการ

4. การใช้ PWM ในการควบคุมความเร็วของมอเตอร์

- PWM (Pulse Width Modulation):เป็นการส่งงานโดยการเปิดปิดสัญญาณไฟฟ้าด้วยความถี่สูง การปรับสัดส่วนเวลาเปิด (duty cycle) จะเปลี่ยนค่าเฉลี่ยของแรงดันที่ส่งไปยังมอเตอร์
- ผลลัพธ์:
 - Duty cycle สูง (เช่น 80-100%) = มอเตอร์หมุนเร็ว
 - Duty cycle ต่ำ (เช่น 20-40%) = มอเตอร์หมุนช้า

5. การเชื่อมต่อ Motor Driver กับ ESP32ตัวอย่างการเชื่อมต่อ (ใช้ L298N เป็นตัวอย่าง)

- สัญญาณควบคุม:
 - EN1, IN2: ควบคุมทิศทางการหมุนของมอเตอร์
 - ENA: สัญญาณ PWM สำหรับควบคุมความเร็ว (เชื่อมกับช่อง PWM บน ESP32)
- การจ่ายไฟ:
 - Vcc และ GND: เชื่อมต่อกับแหล่งจ่ายไฟที่เหมาะสม (สำหรับมอเตอร์และลอจิก)
 - การเชื่อมต่อกับมอเตอร์:
 - ขา Output ของ Motor Driver (เช่น OUT1, OUT2) เชื่อมต่อกับขาของมอเตอร์

6. ข้อดีตัวอย่างการควบคุมมอเตอร์และใช้งานร่วมกับเซ็นเซอร์ Ultrasonic

คำอธิบายโค้ด

- การตั้งค่าพารามิเตอร์และ PWM: กระพริบรัว ๆ
 - กำหนดพินสำหรับควบคุมทิศทาง (IN1, IN2) และพินสำหรับควบคุมความเร็ว (ENA)
 - ใช้ฟังก์ชัน ledcSetup() และ ledcAttachPin() สำหรับการตั้งค่า PWM บน ESP32
- ฟังก์ชัน measureDistance():
 - ทำการส่งสัญญาณผ่านพิน TRIG_PIN และรับสัญญาณสะท้อนจาก ECHO_PIN
 - คำนวณระยะทางโดยใช้สูตรพื้นฐานจากเวลาที่ใช้ในการเดินทางของเสียง
- ในฟังก์ชัน loop():
 - วัดระยะทางโดยใช้ Ultrasonic sensor
 - หากระยะทางมากกว่าเกณฑ์ที่กำหนด (20 cm) จะสั่งให้มอเตอร์หมุนไปข้างหน้าโดยส่งสัญญาณที่เหมาะสม และใช้ PWM ในการกำหนดความเร็ว
 - หากวัดระยะทางน้อยกว่าเกณฑ์ (มีสิ่งกีดขวาง) จะสั่งให้หยุดมอเตอร์

สรุป

- Motor Driver และ H-Bridge:ทำงานร่วมกันเพื่อควบคุมทิศทางและความเร็วของมอเตอร์ DC โดยใช้สัญญาณจาก ESP32
- การใช้ PWM:ช่วยในการปรับความเร็วของมอเตอร์โดยการเปลี่ยน duty cycle ของสัญญาณควบคุม
- การผสานเซ็นเซอร์:ใช้เซ็นเซอร์ Ultrasonic เพื่อตรวจจับวัตถุและสั่งให้มอเตอร์หยุดหรือเมื่อตรวจพบสิ่งกีดขวาง ช่วยเพิ่มความปลอดภัยในการใช้งาน

ESP32 ที่ LED ใช้ค่า 0-255 เพราะใช้ PWM แบบ 8-bit

0 = ดับ

255 = สว่างสุด

ค่าระหว่างนั้น = ความสว่างมากน้อยต่างกัน

เหตุผล: 8-bit มีได้ 256 ค่า เลขนับตั้งแต่ 0 ถึง 255

หลักการ:

ESP32 ไม่ได้อัดแรงดันตรง ๆ แต่มักใช้ PWM คือเปิด-ปิดไฟเร็วมาก

ยิ่งเปิดนานใน 1 รอบมาก → LED ยิ่งสว่าง

ตัวอย่าง:

0 = เปิด 0% → ดับ

128 ≈ เปิด 50% → สว่างกลาง

255 = เปิด 100% → สว่างสุด

ประโยชน์ของทั้งสอง:

ค่า LED 0-255 ใน ESP32 มักมาจากการควบคุมแบบ PWM ความละเอียด 8-bit ซึ่งมีค่าได้ 256 ระดับตั้งแต่ 0 ถึง 255 โดย 0 คือดับ และ 255 คือสว่างสุด ค่าระหว่างนั้นใช้กำหนด duty cycle เพื่อปรับความสว่างของ LED